

Documentación de ShareUrCar

Introducción y Tecnologías

Empezamos este documento explicando la base de nuestro proyecto. Para el desarrollo hemos decidido utilizar dos tecnologías que hemos visto durante el curso, usando **Spring Boot** para todo el backend y **Flutter** para la aplicación frontend. Tomamos esta decisión porque nos sentimos cómodos con ellas y nos permiten tener el código bien separado entre la interfaz que ve el usuario y la lógica del servidor.

La estructura del proyecto se divide en dos carpetas principales: **shareurcar-backend** con toda la lógica del servidor en Java, y **shareurcar_app_frontend** con el código de la aplicación móvil en Dart.

Nombre	Fecha de modificación	Tipo	Tamaño
Documentacion	26/05/2026 20:48	Carpeta de archivos	
shareurcar_app_frontend	26/05/2026 12:42	Carpeta de archivos	
shareurcar-backend	25/05/2026 12:50	Carpeta de archivos	

Cambios que se han hecho

Durante las primeras semanas nos fuimos dando cuenta de que nuestro planteamiento inicial de la base de datos se quedaba corto para una aplicación de viajes compartidos real. Al principio pensábamos manejar las rutas de forma sencilla con nombres de ciudades, pero pronto vimos que necesitábamos guardar la latitud y longitud exacta de los orígenes y destinos. Esto nos obligó a modificar las entidades principales para poder guardar estas coordenadas y calcular distancias de forma precisa. En concreto, tuvimos que añadir las columnas **origin_lat**, **origin_lng**, **dest_lat** y **dest_lng** directamente en nuestra tabla principal de **routes**.

Otra cosa importante que tuvimos que reestructurar en la base de datos sobre la marcha fue la gestión de los pasajeros. Nos dimos cuenta de que, si un viaje tenía varios pasajeros, no podíamos guardar el estado de confirmación del viaje directamente en la ruta general porque cada persona llega o confirma en momentos distintos. Para esto teníamos dos tablas: **travel_groups**, que relaciona la ruta con el conductor y sus horarios, y **group_passengers**, que vincula a cada usuario individual con ese grupo de viaje. De esta forma, cada usuario tiene su propio estado de confirmación en la base de datos, lo que hace

el sistema mucho más seguro y evita que el conductor pueda finalizar un viaje si los pasajeros reales no han confirmado su llegada. A nivel de base de datos, esto supuso modificar la tabla añadiendo la columna booleana **driver_confirmed** y la columna **status** en la tabla `routes`, además de añadir la columna **confirmed** en la tabla `group_passengers`.

Respecto a los cambios que fuimos haciendo sobre la marcha, uno de los más importantes fue eliminar la tabla **passengers** que teníamos planteada al principio. La descartamos porque toda esa funcionalidad quedó cubierta de forma mucho más completa con la combinación de **travel_groups** y **group_passengers**, que además nos da el estado individual de confirmación de cada persona. Mantener las dos sería tener información duplicada sin ningún propósito.

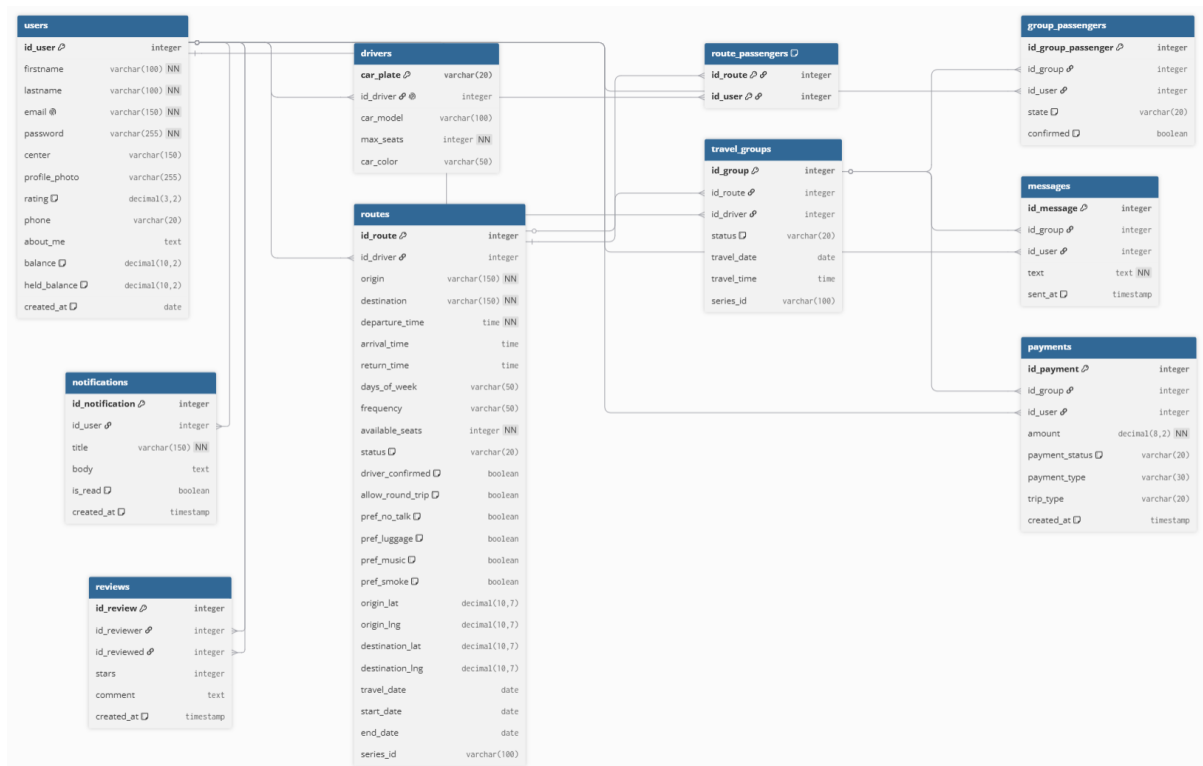
También añadimos columnas a la tabla **users** que no teníamos en el diseño original: **balance** y **held_balance**. El balance es el saldo total del usuario en la plataforma, y el **held_balance** es la cantidad que está retenida en pagos pendientes de confirmar. Esta separación fue necesaria para que el usuario pueda ver en todo momento cuánto dinero tiene disponible de verdad, sin contar lo que ya está comprometido en viajes futuros.

Otro cambio grande fue añadir la columna **series_id** en la tabla `routes`. Esto lo necesitamos para poder agrupar todas las instancias de una ruta semanal bajo un mismo identificador (un grupo de viaje). Cuando un conductor crea una ruta recurrente de lunes a viernes durante un mes, se generan varias filas en la tabla `routes` (una por cada día), pero todas comparten el mismo **series_id**. Gracias a esto podemos ofrecer al pasajero la opción de unirse a toda la serie de una sola vez.

Finalmente, añadimos la tabla entera de **notifications** que no estaba en el diseño original. Esta tabla tiene columnas clave como **id_user**, **title**, **body**, **is_read** y **created_at**, y la usamos para notificar a los usuarios de eventos importantes: cuando un viaje se confirma, cuando alguien reporta una incidencia, o cuando el conductor cancela una ruta. El sistema de incidencias que mencionábamos en la documentación anterior lo acabamos implementando directamente sobre esta tabla de notificaciones, en lugar de crear una tabla separada de incidents, porque para nuestro caso de uso era suficiente con guardar la notificación y que quede registrado quién la envió.

Base de datos – Diagrama final

El diagrama entidad-relación definitivo de la base de datos quedó con las siguientes tablas. Lo ponemos aquí para que quede claro el modelo final que se usó durante el desarrollo y las pruebas:



Las relaciones principales son las siguientes: un **user** puede ser **driver** si registra su vehículo. Un **driver** puede crear varias **routes**. Cada **route** tiene un **travel_group** asociado que se crea automáticamente al publicar la ruta. Los usuarios que se unen a una ruta aparecen como **group_passengers** dentro de ese grupo, y cada uno tiene su propio pago PENDING en la tabla **payments**. Cuando todos confirman el viaje, los pagos pasan a COMPLETED y el conductor recibe el dinero. Las **notifications** se generan en eventos y cada una pertenece a un único usuario.

Arquitectura del backend

El backend sigue una arquitectura en capas típica de Spring Boot. Tenemos los controllers que reciben las peticiones HTTP, los services donde está toda la lógica de negocio, los repositories que se comunican con la base de datos usando Spring Data JPA, y los models que representan las tablas. Además, añadimos una carpeta util para clases de apoyo (un Route Mapper para ahorrar líneas de código) y exception para los errores.

Gestión de rutas — RouteService

Este es el servicio más grande del proyecto porque concentra toda la lógica económica de la aplicación. Tiene varios métodos privados auxiliares que reutilizamos en distintas partes: el método **round2()** para redondear siempre a dos decimales, **bal()** y **held()** para leer el saldo de un usuario sin tener que comprobar nulls cada vez, y **priceFor()** que calcula el precio de una ruta según si es ida o ida y vuelta.

```
// helpers
private static double round2(double x) { return Math.round(x * 100.0) / 100.0; }

private static double bal(User u) { return u.getBalance() != null ? u.getBalance() : 0.0; }

private static double held(User u) { return u.getHeldBalance() != null ? u.getHeldBalance() : 0.0; }
```

La función más compleja de este servicio es **calculateTripPrice()**, que calcula el precio justo del trayecto. Para hacerlo usamos la fórmula de **Haversine** (explicada más adelante) para obtener los kilómetros reales entre origen y destino, y luego aplicamos un factor corrector según si la distancia es urbana o interurbana, porque los trayectos cortos dentro de ciudad tienen un coste por kilómetro más alto. El precio final también varía según el número de pasajeros: a más gente en el coche, menos paga cada uno, usando un array: 0.425, 0.550, 0.675 y 0.80 como porcentajes de reparto.

```
private double calculateTripPrice(Route route, int passengers) {
    double km = haversine(route.getOriginLat(), route.getOriginLng(),
        route.getDestinationLat(), route.getDestinationLng());

    // Factor corrector por distancia corta
    if (km < 3) km *= 1.80;
    else if (km < 5) km *= 1.40;
    else if (km < 10) km *= 1.25;
    else km *= 1.15;

    double[] pcts = {0.425, 0.550, 0.675, 0.80};
    double pct = pcts[Math.min(passengers - 1, 3)];
    double price = (km * 0.40 * pct / passengers) * 1.20;

    double min = km <= 5 ? 1.20 : km <= 9 ? 1.75 : km <= 14 ? 2.00 : 2.50;
    return round2(Math.max(price, min));
}
```

Para las rutas recurrentes, **createRoutes()** recibe un DTO con las fechas de inicio y fin y los días de la semana seleccionados. El método itera día a día entre esas fechas, comprueba si el día de la semana coincide con alguno de los seleccionados usando el Map estático **DAY_MAP**, y si es así crea una instancia de Route con ese día concreto como travel_date. Todas las instancias generadas comparten el mismo series_id que se genera con un random.

```
@Transactional 1 usage Alejandro Sánchez Cordero +1
public List<Route> createRoutes(RouteCreateDTO dto) {
    if (dto.getIdDriver() == null) {
        throw new IllegalArgumentException("Driver requerido");
    }
    if (!driverRepository.existsByIdDriver(dto.getIdDriver())) {
        throw new UserIsNotDriverException(
            "Este usuario no está registrado como conductor (falta el coche)");
    }
    if (dto.getAvailable_seats() == null || dto.getAvailable_seats() <= 0) {
        throw new IllegalArgumentException("La cantidad de sitios es inválida");
    }

    Route base = buildRouteFromDto(dto);
    String seriesId = UUID.randomUUID().toString();
    List<Route> saved = new ArrayList<>();

    if ("semanal".equalsIgnoreCase(dto.getFrequency())) {
        List<DayOfWeek> validDays = parseDaysOfWeek(dto.getDays_of_week());
        LocalDate current = dto.getStart_date();
        LocalDate end = dto.getEnd_date();
        while (!current.isAfter(end)) {
            if (validDays.contains(current.getDayOfWeek())) {
                Route instance = cloneRoute(base);
                instance.setTravel_date(current);
                instance.setSeriesId(seriesId);
                instance.setDriverConfirmed(false);
                saved.add(routeRepository.save(instance));
            }
            current = current.plusDays(daysToAdd: 1);
        }
    } else {
        base.setSeriesId(seriesId);
        saved.add(routeRepository.save(base));
    }

    saved.forEach(this::createGroupForRoute);
    return saved;
}
```

El método **joinSingleRouteInternal()** es el núcleo de la unión a una ruta. Crea el grupo de viaje si no existe todavía, añade al usuario como GroupPassenger, crea el pago en estado PENDING y retiene el importe del saldo del usuario. Este método lo reutilizan tanto **joinRoute()** para una sola ruta como **joinRoutes()** para el multi-join y **joinSeries()** para unirse a toda la serie.

```
private void joinSingleRouteInternal(Route route, User user, boolean roundTrip) {
    double amount = round2(priceFor(route, roundTrip));

    TravelGroup group = travelGroupRepository.findByIdRoute(route.getIdRoute())
        .orElseGet(() -> {
            TravelGroup g = new TravelGroup();
            g.setIdRoute(route.getIdRoute());
            g.setIdDriver(route.getIdDriver());
            g.setStatus("ACTIVE");
            g.setTravelDate(route.getTravel_date());
            g.setTravelTime(route.getDeparture_time());
            return travelGroupRepository.save(g);
        });

    if (groupPassengerRepository.findByIdGroupAndIdUser(group.getIdGroup(), user.getIdUser()).isEmpty()) {
        GroupPassenger gp = new GroupPassenger();
        gp.setIdGroup(group.getIdGroup());
        gp.setIdUser(user.getIdUser());
        gp.setState("ACTIVE");
        gp.setConfirmed(false);
        groupPassengerRepository.save(gp);
    }

    Payment payment = new Payment();
    payment.setIdGroup(group.getIdGroup());
    payment.setIdUser(user.getIdUser());
    payment.setAmount(round2(amount));
    payment.setPaymentStatus("PENDING");
    payment.setTripType(roundTrip ? "ROUND_TRIP" : "ONE_WAY");
    payment.setCreatedAt(LocalDate.now());
    payment.setPaymentType("TRIP_PAYMENT");
    paymentRepository.save(payment);

    user.setHeldBalance(round2(x held(user) + amount));
    userRepository.save(user);

    route.getPassengers().add(user);
    route.setAvailable_seats(route.getAvailable_seats() - 1);
    routeRepository.save(route);
}
```

Cuando todos los participantes pulsán el botón de confirmar, **checkAllConfirmed()** devuelve true y se llama a **settleRoutePayments()**. Este método recalcula el precio real con el número final de pasajeros, devuelve la diferencia si alguien pagó de más, descuenta el importe definitivo del saldo de cada pasajero, y transfiere el total al saldo del conductor.

Sistema de notificaciones — NotificationService

Este servicio se encarga de crear y gestionar las notificaciones de los usuarios. El método **reportarIncidencia()** recibe el ID de la ruta, el ID de quien reporta, y el mensaje. No cancela la ruta ni devuelve saldos: simplemente envía la notificación a los afectados y deja la ruta activa. Si quien reporta es el conductor, la notificación llega a todos los pasajeros. Si es un pasajero, solo le llega al conductor. Esta decisión la tomamos porque anular un viaje entero por la queja de una sola persona no tiene sentido, y en una versión real habría un equipo de soporte revisando cada caso, así queda como simulado un sistema de incidencias pendientes de revisión humana real.

Manejo de errores — GlobalExceptionHandler

Para no repetir el mismo bloque de ResponseEntity en cada handler, extrajimos un método privado error (int code, String msg, HttpStatus status) que construye la respuesta. Esto redujo el handler a la mitad de líneas manteniendo exactamente el mismo comportamiento.

Fórmula de Haversine

Sobre la parte de buscar rutas, una vez que tenemos las coordenadas reales, llegó una de las partes más complejas del código de nuestro backend. Tuvimos que buscar e implementar una función matemática bastante larga conocida como la **fórmula de Haversine**, que la usan otras aplicaciones de carpooling. Esta función, que metimos directamente en las queries de nuestro repository de rutas, se encarga de **calcular la distancia** real en línea recta entre dos puntos **usando su latitud y longitud** (las que tuvimos que añadir). La metimos en dos sitios: en el RouteRepository como una @Query nativa de SQL para filtrar rutas cercanas directamente en la base de datos, y en RouteService como método Java privado **haversine()** para calcular el precio según los kilómetros reales del trayecto.

```
@Query(value = """
    SELECT * FROM routes r
    WHERE\s
    (6371 * acos(cos(radians(:oLat)) * cos(radians(r.origin_lat))\s
    * cos(radians(r.origin_lng) - radians(:oLng))\s
    + sin(radians(:oLat)) * sin(radians(r.origin_lat)))) <= :radius
    AND\s
    (6371 * acos(cos(radians(:dLat)) * cos(radians(r.destination_lat))\s
    * cos(radians(r.destination_lng) - radians(:dLng))\s
    + sin(radians(:dLat)) * sin(radians(r.destination_lat)))) <= :radius
    ORDER BY (6371 * acos(cos(radians(:oLat)) * cos(radians(r.origin_lat))\s
    * cos(radians(r.origin_lng) - radians(:oLng))\s
    + sin(radians(:oLat)) * sin(radians(r.origin_lat)))) ASC
    \s""", nativeQuery = true)

List<Route> findNearbyRoutes(
    @Param("oLat") Double oLat,
    @Param("oLng") Double oLng,
    @Param("dLat") Double dLat,
    @Param("dLng") Double dLng,
    @Param("radius") Double radius
);
```

Gracias a esto, cuando un pasajero busca un viaje, el servidor calcula las distancias y le devuelve solo las rutas que pasan cerca de su ubicación en un radio de 2.5 kilómetros. Este radio lo elegimos porque en Alicante, que es la ciudad donde se centra

la app, cubre bien los trayectos típicos de casa al centro educativo sin devolver resultados demasiado lejanos.

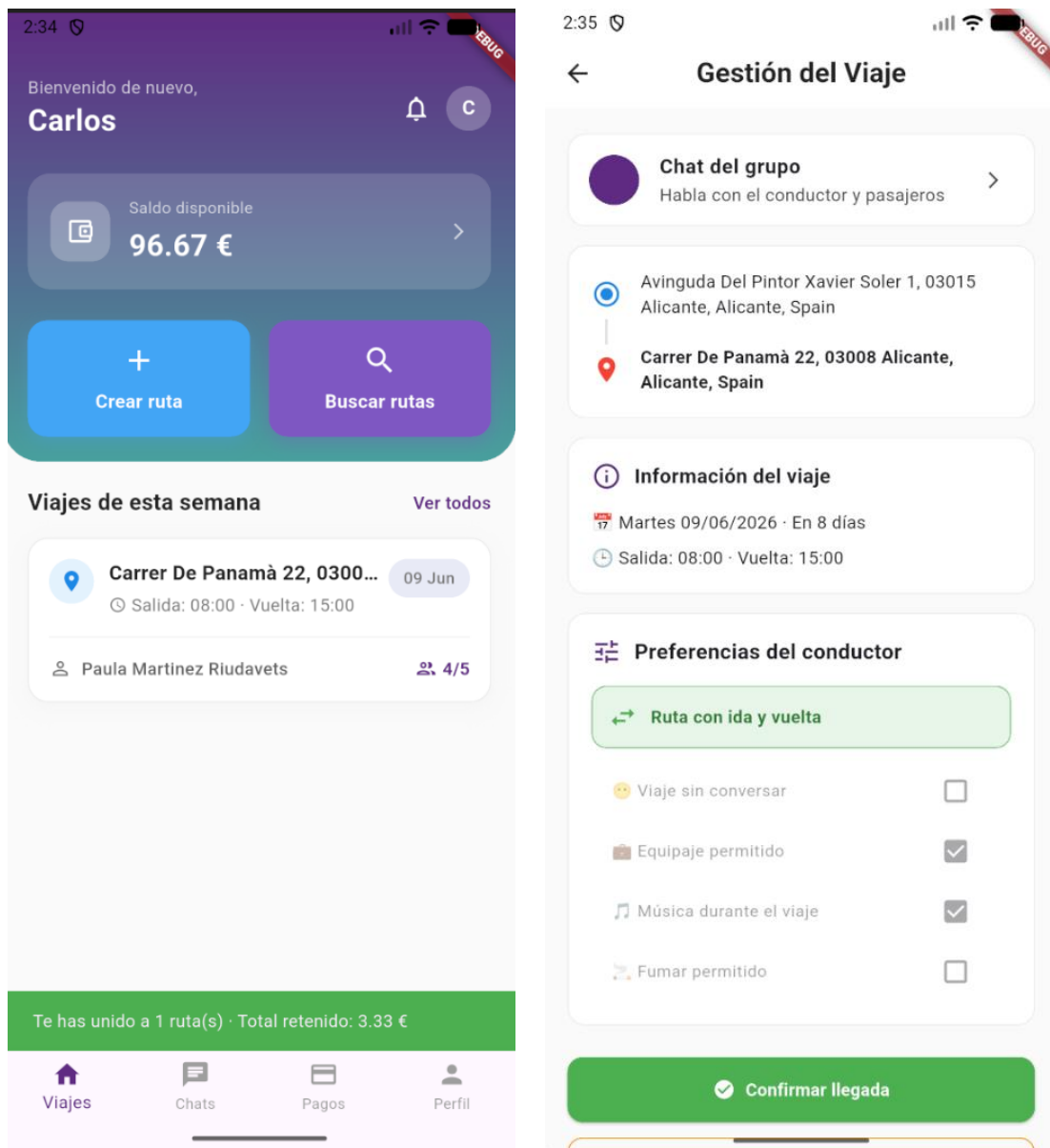
Flujo de la aplicación

Respecto al flujo de la aplicación, el usuario interactúa con varias pestañas principales que agrupan toda esta lógica. En las pantallas de crear y buscar ruta que se accede a través del menú de inicio, implementamos una integración con los servicios de **Mapbox**. Usamos un **token** de acceso para que los usuarios puedan escribir el nombre de una calle o un centro educativo y el sistema les devuelva las sugerencias y coordenadas exactas. En esta parte del código del frontend tuvimos que investigar cómo gestionar para ocultar el token de seguridad porque al hacer commit daba un aviso. Para ello pusimos el token en un archivo aparte llamado **.env** que es donde suele meterse por lo que hemos visto, y metiéndolo en el **.gitignore** ya nos dejaba. Al final conseguimos que el mapa y las direcciones funcionen en tiempo real, y hemos puesto para las sugerencias estén centradas en las coordenadas de Alicante para que no saliesen calles de otras ciudades.

Pantalla Inicio (Viajes)

La experiencia del usuario se centra en la pantalla de inicio, donde tenemos los botones principales para crear y buscar rutas, junto con una lista de los viajes de la semana actual. Solo mostramos los viajes de los próximos 7 días a partir del más próximo para no sobrecargar la pantalla, y los viajes completados o cancelados no aparecen en esta lista. Si el usuario mantiene pulsado sobre una tarjeta de una ruta cancelada, le aparece un diálogo para quitarla de la vista sin borrarla de la base de datos, que seguirá siendo accesible desde el historial de chats.

En la cabecera aparece el saldo disponible del usuario (balance menos held_balance) y la campana de notificaciones con un badge rojo que muestra cuántas tiene sin leer. Al pulsar la campana navega a la pantalla de notificaciones donde se marcan todas como leídas automáticamente.

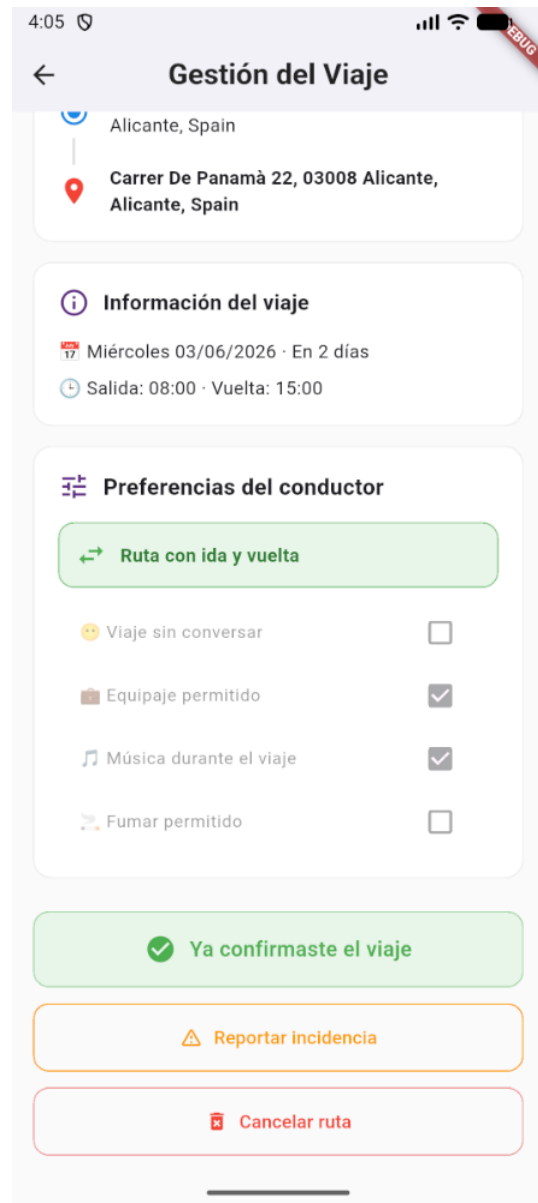
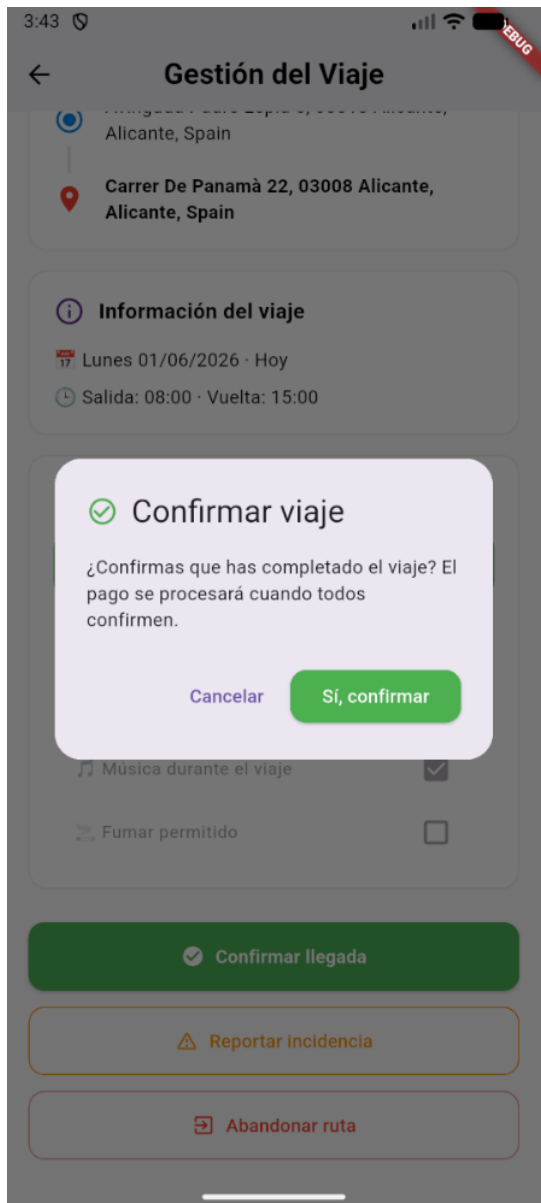


Al pulsar sobre la tarjeta de un viaje activo el usuario pasa a la pantalla de gestión del viaje. Esta pantalla funciona de forma distinta según el rol: si eres el conductor ves el botón de confirmar que dice "Confirmar viaje realizado", y si eres pasajero dice "Confirmar llegada". El botón cambia a un banner verde de "Ya confirmaste" en cuanto pulsas, para evitar que pulses dos veces. El viaje no se cierra hasta que todos los participantes han confirmado: el backend comprueba esto en `checkAllConfirmed()` y solo llama a `settleRoutePayments()` cuando la condición se cumple.

También implementamos una red de seguridad automática en el backend usando una tarea programada con `@Scheduled`. Esta función se ejecuta sola cada hora en punto, compara la hora actual con la base de datos y busca todas las rutas que siguen pendientes pero cuya hora de llegada ya ha pasado. Si encuentra alguna, el sistema asume que el viaje terminó y le cambia el estado a `COMPLETED` automáticamente.

```
@Scheduled(cron = "0 0 * * *") every hour & Alejandro Sánchez Cordero
public void autoCompleteOldRoutes() {
    LocalDateTime now = LocalDateTime.now();
    List<Route> rutasAntiguas = routeRepository.findPendingRoutesWhereArrivalTimePassed(now);

    for (Route r : rutasAntiguas) {
        r.setStatus("COMPLETED");
        routeRepository.save(r);
    }
}
```



Pantalla Crear Ruta

Pantalla de crear ruta donde se muestran **sugerencias**, y el resto de la pantalla donde puede verse el sistema de elección de días de ruta:

Publicar un viaje

Recorrido

Origen
Desde dónde sales?

Destino
¿A dónde vas?

¿Cuándo viajas?

Un día concreto **Viaje recurrente**

Selecciona los días:

L M X J V
S D

Inicia Seleccionar Termina Seleccionar

Detalles

Hora de salida
8:00 AM

Asientos libres
3

Permitir ida y vuelta
Los pasajeros podrán reservar regreso

Permitir ida y vuelta
Los pasajeros podrán reservar regreso

Puntos de recogida (opcional)
Añadir dirección...

Preferencias del viaje

- Viaje sin conversar
- Equipaje permitido
- Música durante el viaje
- Fumar permitido

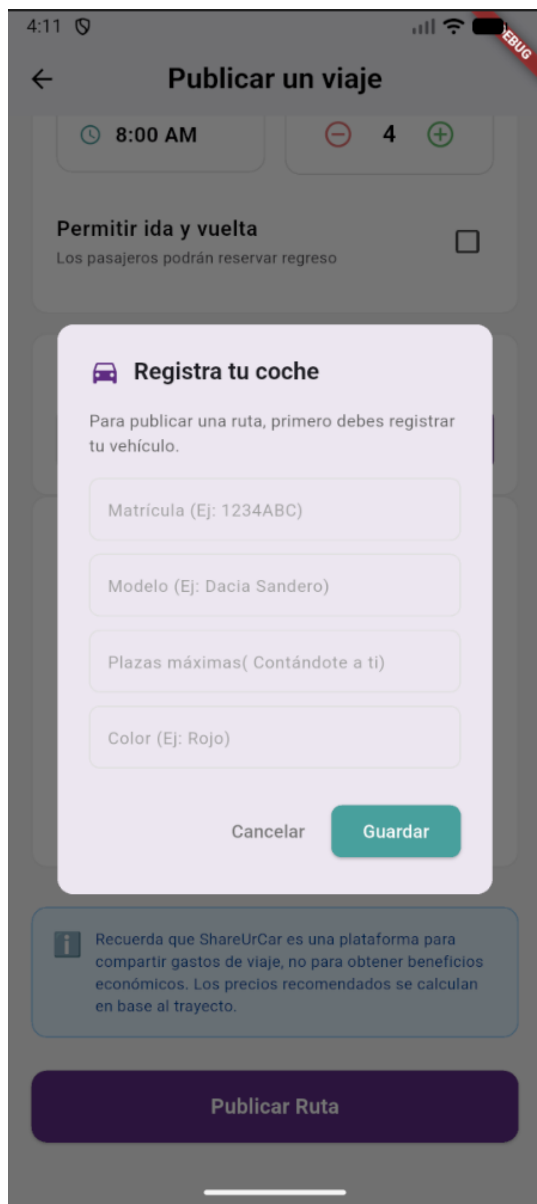
Recuerda que ShareUrCar es una plataforma para compartir gastos de viaje, no para obtener beneficios económicos. Los precios recomendados se calculan en base al trayecto.

Publicar Ruta

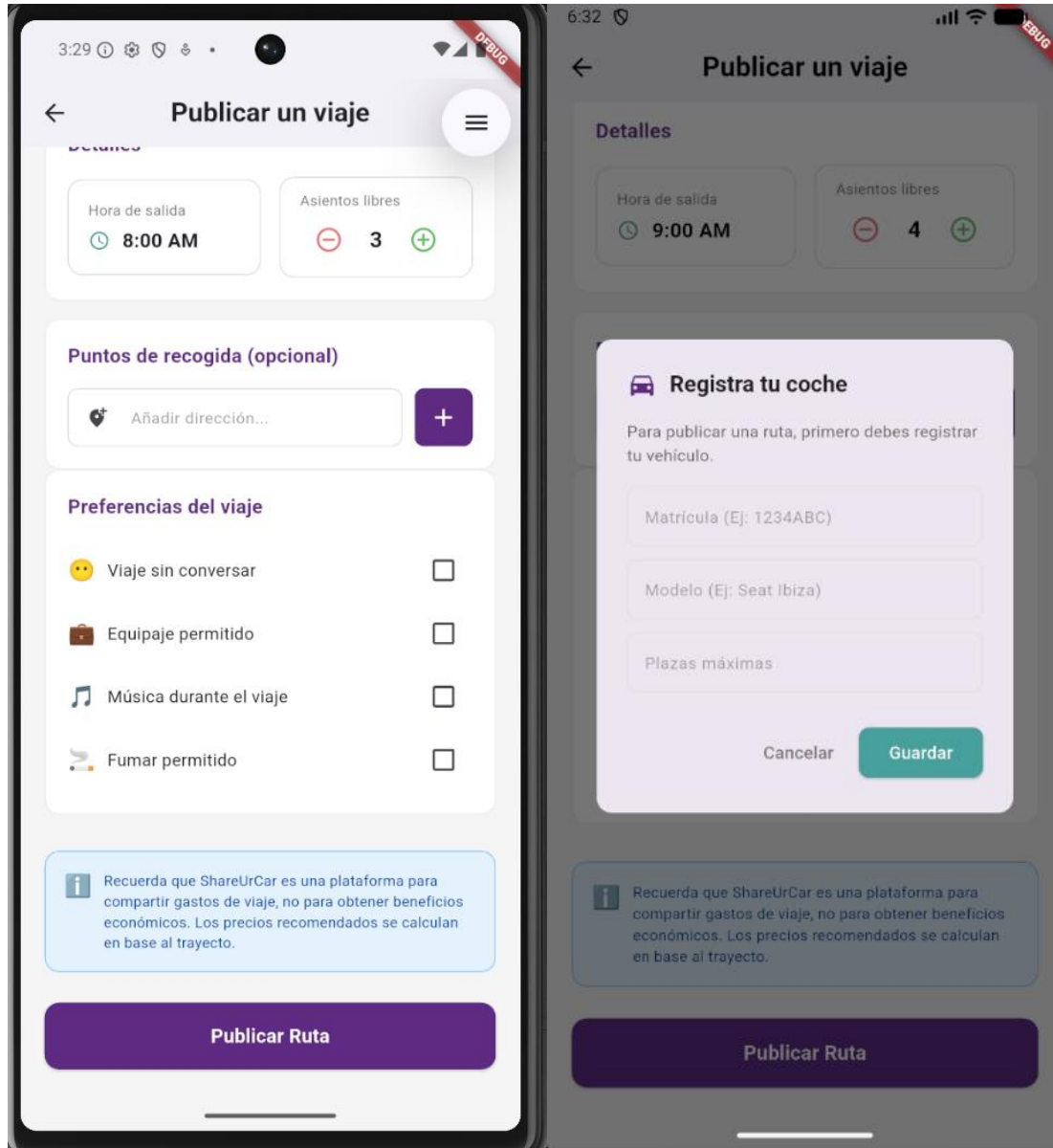
En el código del frontend, CreateRouteScreen usa un **Timer** con debounce de 600 milisegundos para las sugerencias de dirección. Esto significa que no lanzamos una petición a Mapbox cada vez que el usuario escribe una letra, sino que esperamos a que deje de escribir durante más de medio segundo. Sin esto estaríamos haciendo decenas de peticiones por segundo y agotaríamos el límite de la API gratuita muy rápido.

Cuando el usuario le da a "Publicar Ruta" y no tiene vehículo registrado, le mostramos un diálogo para registrar su coche en ese momento. Si lo guarda correctamente, la petición POST/drivers crea el Driver en el backend y ya puede publicar la ruta sin salir

de la pantalla. Si intenta publicar sin registrar el coche, el backend lanza `UserIsNotDriverException` y el frontend muestra el diálogo automáticamente.



Las **preferencias, puntos de recogida y el botón de publicar ruta** se ven como en la imagen de la izquierda, y luego, el usuario al crear una ruta si NO tiene registrado un vehículo no puede crear la ruta porque todavía no se ha convertido en un **Driver**. Para facilitar esto para el usuario en el momento que le dé a publicar le saldrá una ventana para rellenar los datos de su coche, como se puede ver en la derecha:

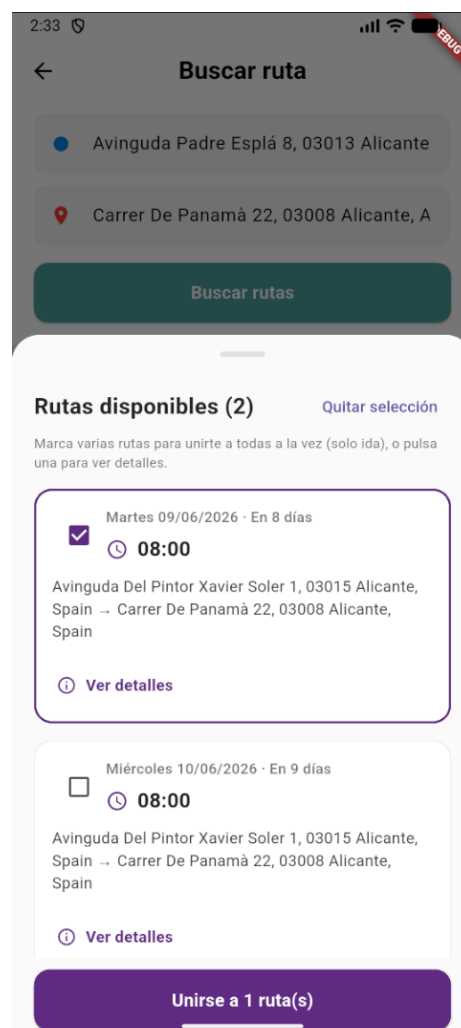
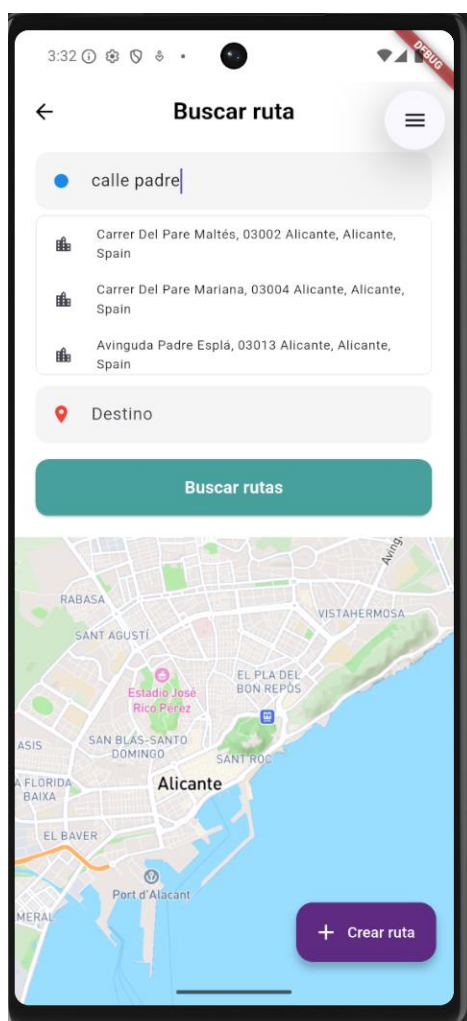


Pantalla Buscar Ruta

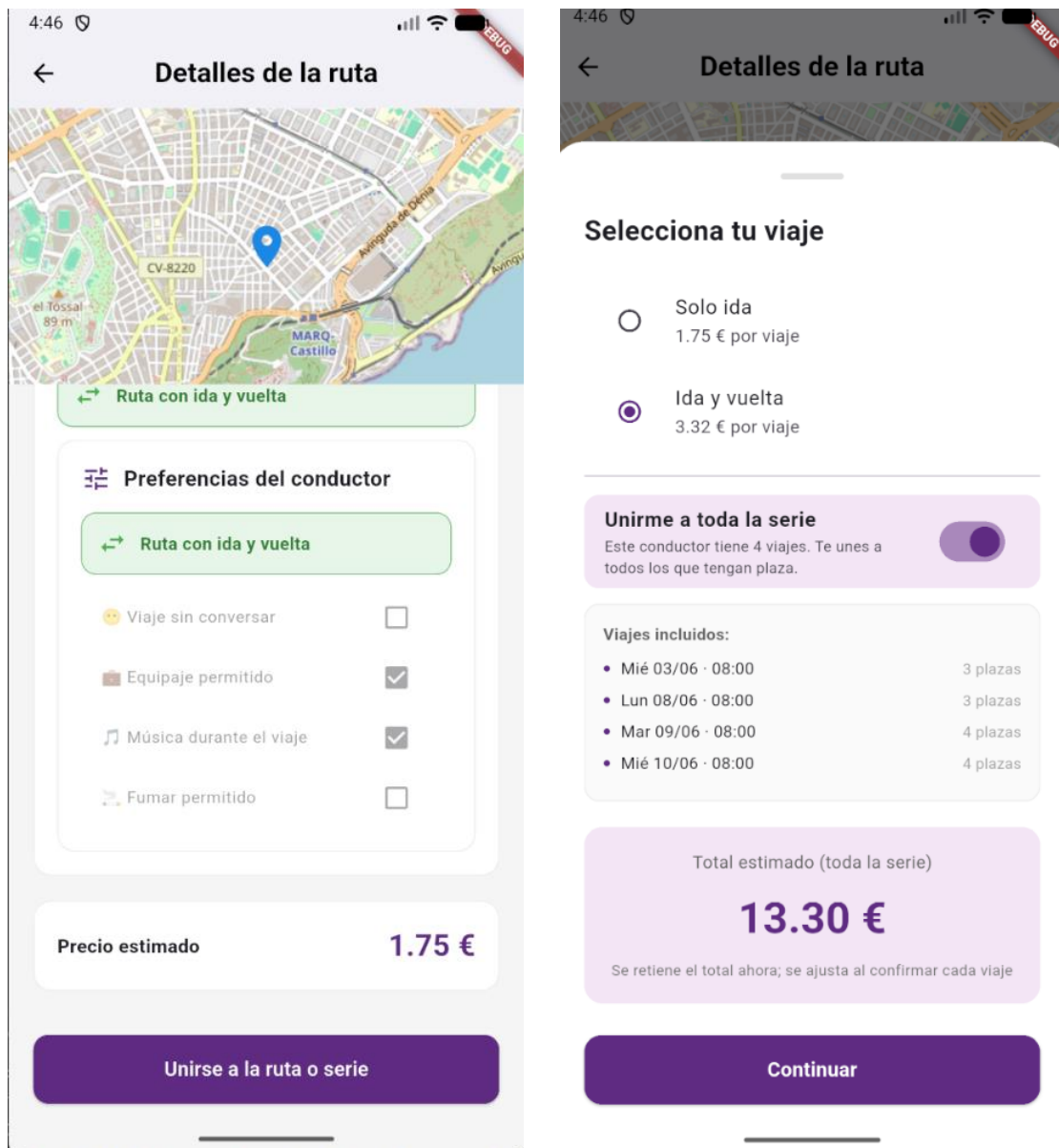
El mapa interactivo de Mapbox se ve así en la pestaña de buscar rutas. Podemos introducir origen y destino mediante sugerencias centradas en las coordenadas de Alicante, y al pulsar "Buscar rutas" el backend devuelve los resultados aplicando la fórmula de Haversine.

Los resultados se muestran en un bottom sheet con varias opciones para el usuario. El flujo de unirse a rutas tiene dos caminos distintos dependiendo de lo que quiera hacer:

Camino 1 — Selección múltiple: El usuario puede marcar varios resultados con los checkboxes (incluso de conductores distintos) y pulsar "Unirse a N rutas". Le aparece primero un diálogo donde ve la lista de rutas que ha seleccionado y puede elegir si quiere solo ida o ida y vuelta. Al pulsar continuar le sale la confirmación final antes de que se ejecute el cobro. Si no tiene saldo suficiente, en lugar del error genérico le aparece un diálogo con el botón de "Gestionar saldo" que le lleva directamente a la pantalla de pagos.



Camino 2 — Ver detalles: Si el usuario pulsa "Ver detalles" en una ruta concreta, va a RouteDetailsScreen donde puede ver toda la información del viaje: mapa, horario, preferencias del conductor y precio estimado. Si la ruta pertenece a una serie recurrente, le aparece un banner con el número total de viajes de la serie y puede activar el switch de "Unirme a toda la serie" para unirse a todos los viajes disponibles de una vez. Al activarlo se despliega la lista con todas las fechas y horas incluidas. En ambos casos (una ruta o la serie) antes de ejecutar el cobro le aparece un popup de confirmación con el importe total a retener.

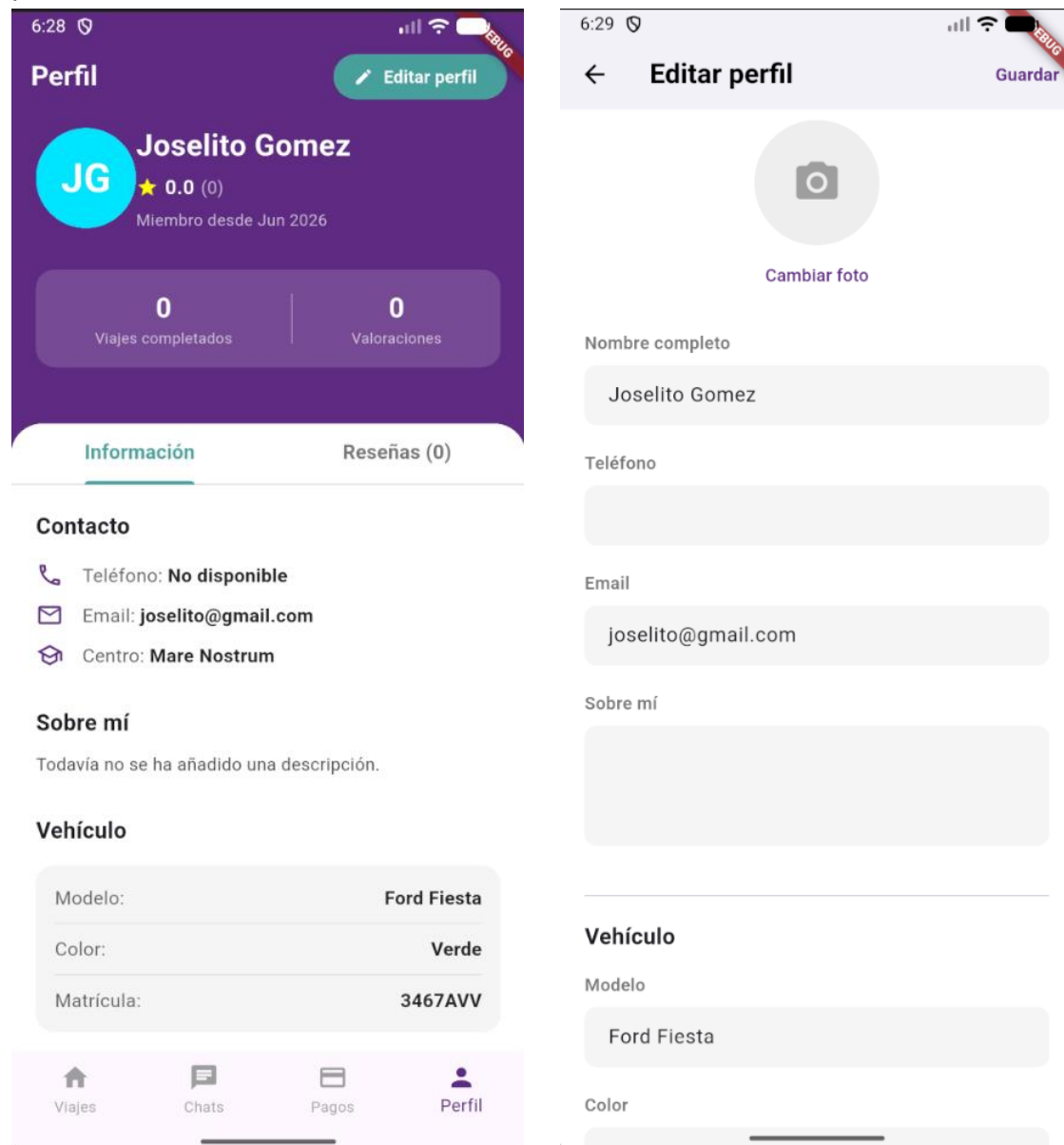


En el backend, el endpoint POST /routes/join-multiple/userID recibe una lista de IDs de rutas y llama a `joinSingleRouteInternal()` en bucle para cada una, validando en cada iteración que el usuario tiene saldo suficiente para el total. Si la validación falla, lanza la excepción antes de ejecutar ningún cobro, así que o se une a todas o no se une a ninguna,

es decir, tiene la anotación `@Transactional`. Para la serie, `joinSeries()` simplemente obtiene todas las rutas del mismo `series_id` y llama a `joinRoutes()` con esa lista.

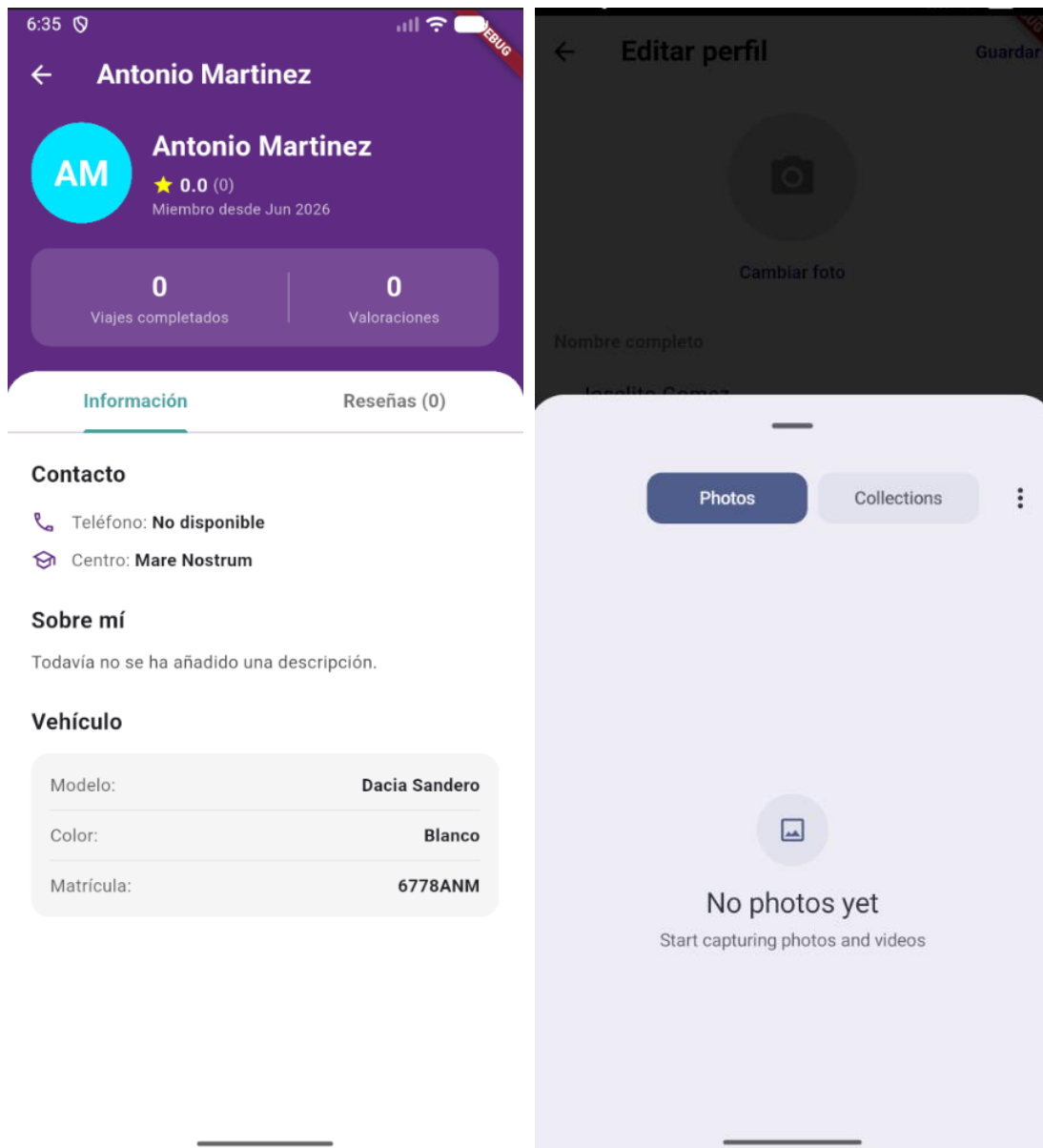
Pantalla Perfil

Después tendríamos la pantalla de perfil donde se puede ver sus **viajes completados**, **reseñas**, **datos de contacto**, un espacio para que el usuario escriba sus **preferencias**, y el **vehículo registrado** (si tiene). Y también la de **editar perfil**, donde el usuario podrá **cambiar su foto** por una de la galería, **editar sus datos** y **añadir un vehículo** para poder crear rutas. Se verían así:



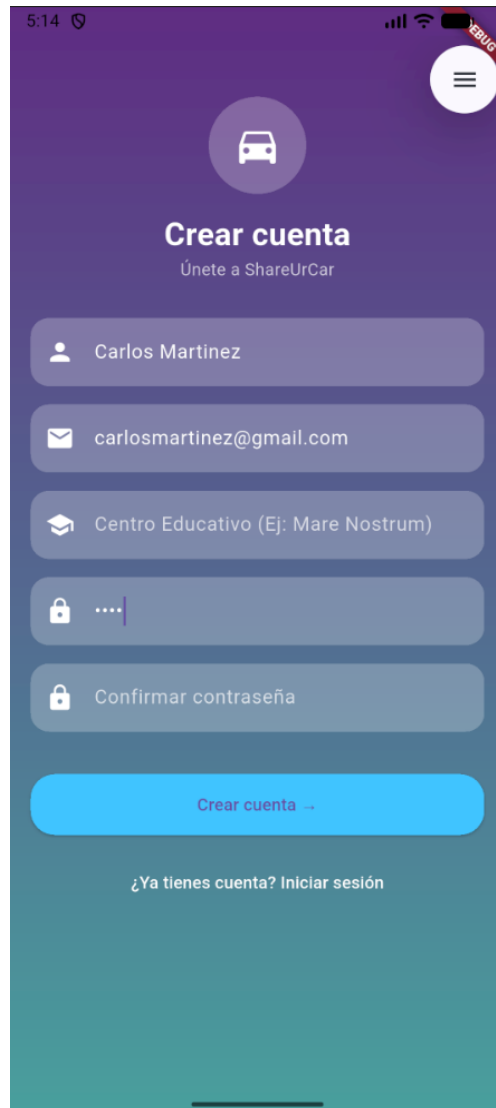
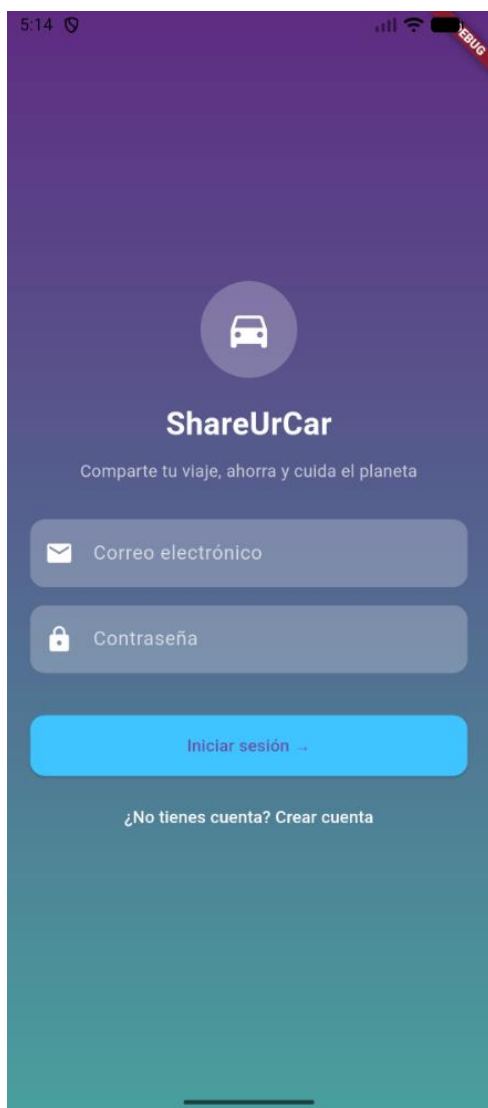
La pantalla de editar perfil permite cambiar la foto de perfil directamente desde la galería del móvil. Usamos el paquete `image_picker` para abrir el selector de imágenes, y una vez seleccionada la convertimos a base64 con `base64Encode` y la enviamos al backend mediante el endpoint `POST /users/userId/photo`. El backend la guarda como texto en la columna `profile_photo` de la tabla `users`. Cuando la foto es una URL externa que empieza por `http` se muestra con `NetworkImage`, y cuando es base64 se decodifica y se muestra directamente.

En la pantalla de perfil público (`PublicProfileScreen`), que se abre al pulsar "Ver perfil" desde los integrantes del chat, se muestran los datos de contacto, la descripción "Sobre mí", el vehículo registrado si tiene, y todas las reseñas que ha recibido ese usuario. La diferencia con el perfil propio es que no aparece el saldo ni el botón de editar.



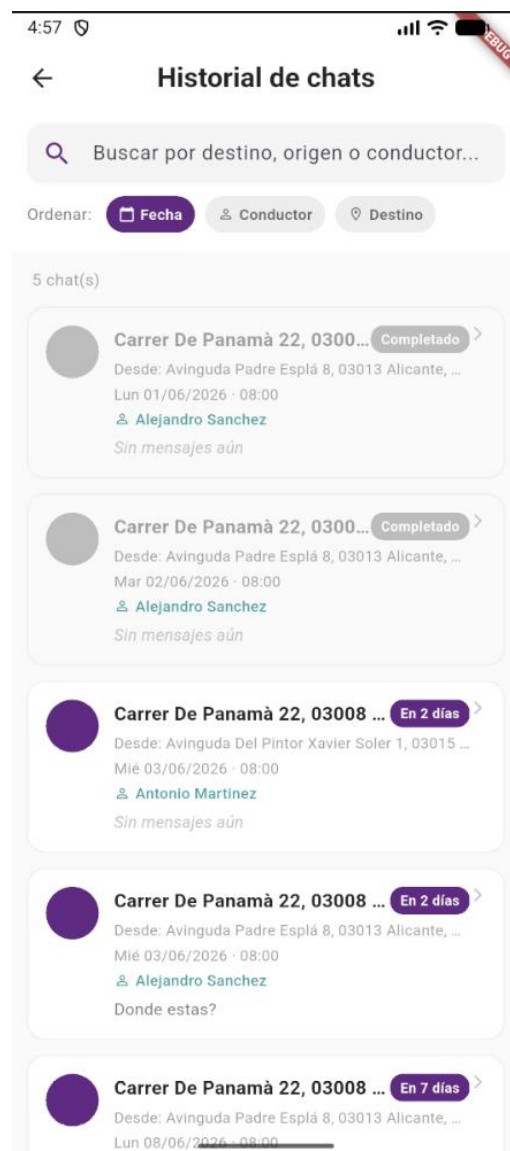
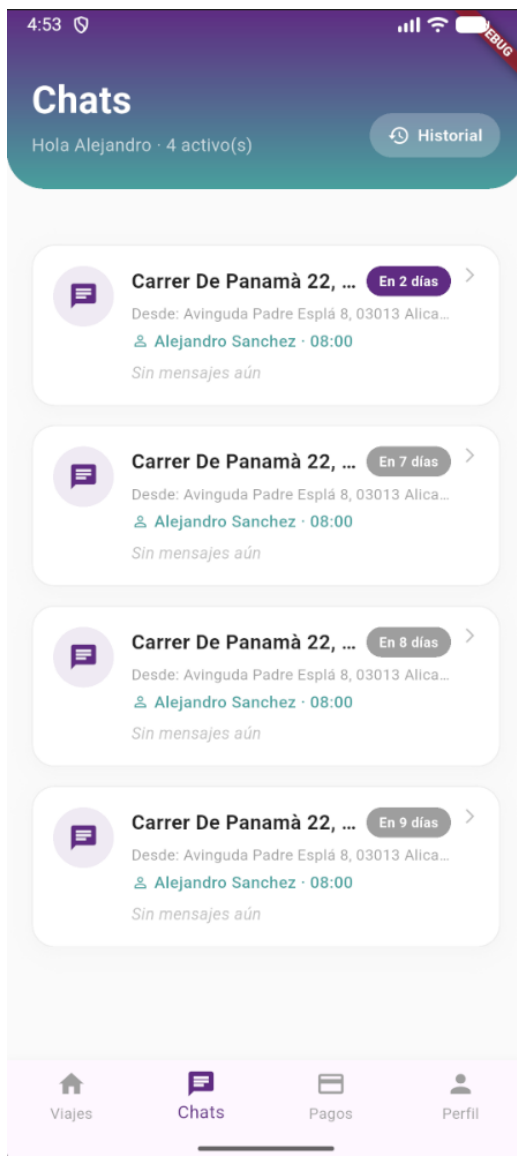
Sistema de autenticación

Para el login y registro usamos el endpoint `/auth/login` y `/auth/register` de `AuthController`. Las contraseñas se almacenan siempre hasheadas con `BCrypt` usando `PasswordEncoder` de `Spring Security`. Al registrarse, si el email ya existe en la base de datos el backend lanza una `exception` con el mensaje "El usuario ya existe".



Pantalla Mensajes

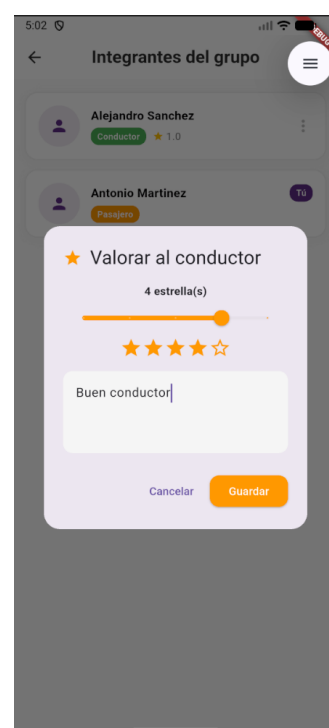
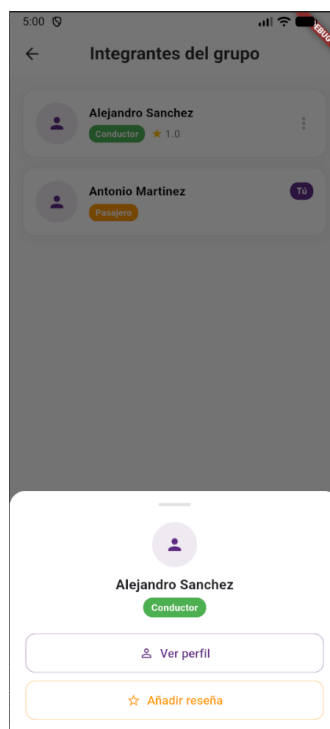
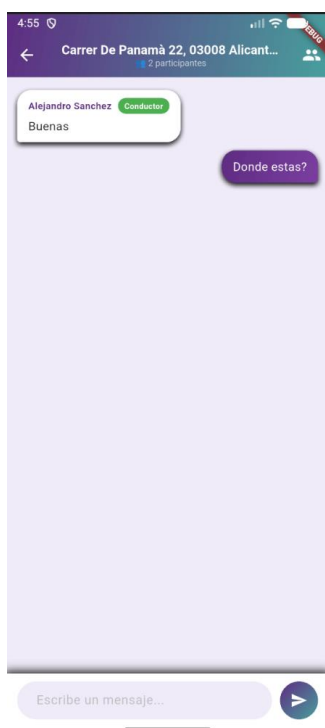
Tenemos una pestaña dedicada a los mensajes. Al unirte o crear una ruta se crea automáticamente un TravelGroup en el backend, que es lo que hace posible el chat. Desde la pestaña de Chats solo se muestran los grupos con rutas activas de la semana actual. Los chats de rutas completadas o pasadas siguen siendo accesibles desde el botón de "Historial", que abre ChatHistoryScreen con un buscador por destino, origen y conductor, y chips para ordenar por fecha, conductor o destino.



Para el chat en tiempo real, implementamos un sistema de polling usando Timer.periodic de Dart. Cada 5 segundos el frontend hace una petición GET al backend y compara el número de mensajes que ha recibido con los que ya tenía. Si el número es distinto, actualiza la lista y hace scroll hasta el último mensaje. Si es igual no hace nada, así que no hay renders

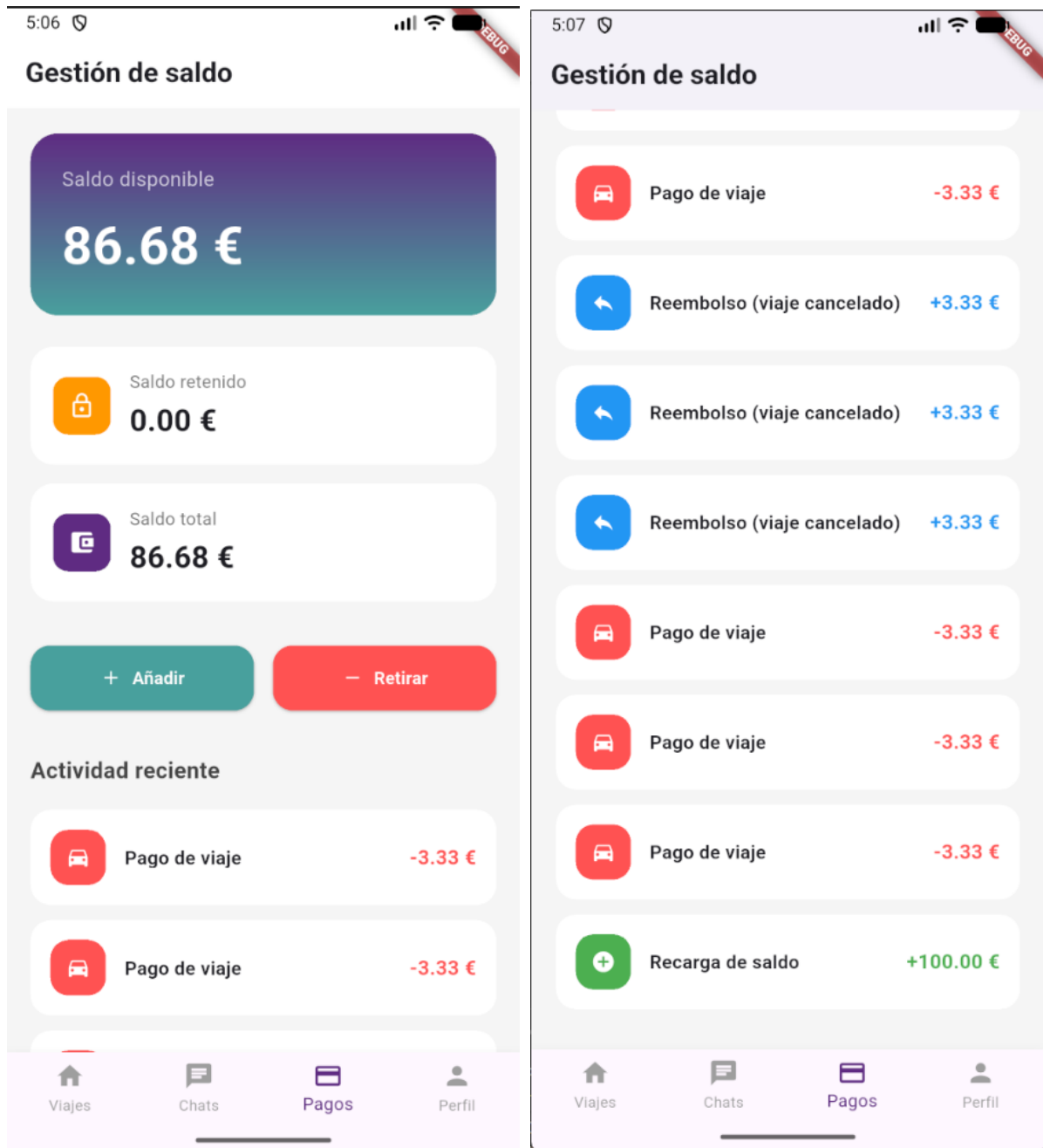
innecesarios. Esto lo pusimos en **fetchMessagesSilent()** separado del **fetchMessages()** normal, para que el refresco automático no muestre ni el icono de carga ni los errores en pantalla.

En la cabecera del chat, al pulsar sobre el nombre del destino o el icono de grupo, navega a GroupMembersScreen. Desde ahí, al pulsar sobre cualquier miembro que no seas tú, aparece un bottom sheet con dos opciones: "Ver perfil" (que abre el perfil público de esa persona con sus estadísticas y reseñas) y "Añadir reseña" (solo disponible si el miembro es el conductor). Decidimos quitar el mensaje privado que teníamos antes porque para eso ya está el chat de grupo, y en una app de carpooling universitaria no tiene mucho sentido añadir mensajes privados.



Pantalla pagos

La pantalla de pagos muestra el saldo disponible del usuario (la diferencia entre balance y held_balance), el saldo retenido en viajes pendientes, y el saldo total. Desde aquí puede añadir o retirar saldo usando los botones correspondientes, con un mínimo de 5 € y un máximo de 500 € por operación que se valida también en el backend.



Debajo de los botones aparece el historial de actividad reciente ordenado de más nuevo a más antiguo. Cada movimiento tiene un icono y color distintos según el tipo:

- Verde con flecha arriba: recarga de saldo (**DEPOSIT**)
- Rojo con coche: pago de viaje (**TRIP_PAYMENT**)
- Verde con monedero: cobro por viaje como conductor (**TRIP_INCOME**)
- Azul con flecha: reembolso (**REFUND**) cuando se abandona una ruta y se cancela
- Verde con hucha: ajuste de precio (**PRICE_ADJUSTMENT**) cuando el precio real fue menor que el retenido

El caso del ajuste de precio ocurre en **settleRoutePayments()** cuando el pasajero pagó más en la reserva de lo que correspondía con el número final de pasajeros. El backend crea un pago nuevo de tipo **PRICE_ADJUSTMENT** con la diferencia y se lo devuelve al usuario.

Cuando un usuario abandona una ruta, el backend crea un pago nuevo de tipo **REFUND** con la fecha actual, de forma que aparece siempre arriba del todo en el historial en lugar de modificar el pago original. El pago original de **TRIP_PAYMENT** queda marcado como **CANCELLED** y también se muestra como reembolso en la actividad.

Refactorización y organización del código

Hacia el final del proyecto hicimos una pasada de limpieza y refactorización en los dos lados del código para dejarlo más organizado y sin repeticiones.

En el backend, extrajimos la conversión de una entidad *Route* a un `Map<String, Object>` a una clase en una carpeta *utils* para llamar a **RouteMapper.toMap()**, porque ese mismo bloque estaba en otras funciones. También extrajimos **refundPendingPayments()** en *RouteService* para que tanto en la función de borrar ruta como el servicio de incidencias puedan reutilizar la lógica de devolver saldos sin duplicar código. El *GlobalExceptionHandler* lo simplificamos con un método privado `error()` que reduce también código.

En el frontend creamos dos archivos. El primero es *app_theme*, que centraliza todos los colores de la aplicación (`kPrimary`, `kSecondary`) y las decoraciones reutilizables como **kGradient**, **kHeaderDecoration()** y **kCardDecoration()**. El segundo es *app_utils* que agrupa todas las funciones de formato de fecha y hora (`formatHora`, `formatFechaCompleta`, `fechaRelativa...`) y otras utilidades como `limpiarDireccion()`, `iniciales()` o `saldoDisponible()`. Estas funciones se usaban en varias pantallas.

También extrajimos el bloque de preferencias del conductor a un widget reutilizable **PreferenciasCard**, que se usa tanto en *RouteDetailsScreen* como en *ActiveTripScreen*.

Para la propagación del estado del usuario entre pestañas, usamos `IndexedStack` en `MainNavigation` para que las pantallas mantengan su estado entre tabs, y un contador `chatsRefresh` que se incrementa cada vez que el usuario toca la pestaña de Chats. `ChatsScreen` detecta el cambio en `didUpdateWidget` y refresca la lista automáticamente, así los chats de rutas nuevas aparecen sin tener que reiniciar la app.